



IndustryForge-27B: A Domain-Enhanced Multimodal Foundation Model for Industrial CAD

Nianchen Deng¹, Jiaxin Ai^{2,1,3}, Tao Hu^{4,1}, Shu Zou^{1,5}, Yurui Dong^{1,6}, Siqi Li^{1,7}, Xinyu Cai¹, Xuemeng Yang¹, Licheng Wen^{1,3}, Hongbin Zhou¹, Hairong Zhang^{6,1}, Pinlong Cai^{1,*}, Botian Shi^{1,3}

¹ Shanghai Artificial Intelligence Laboratory ² Wuhan University ³ Shanghai Innovation Institute
⁴ University of Science and Technology of China ⁵ The Australian National University ⁶ Fudan University ⁷ Zhejiang University

* Corresponding author.

Automating industrial CAD design and manufacturing places distinctive demands on multimodal foundation models: the model must *see* engineering drawings and 3D geometry screenshots, *write* correct parametric-modelling scripts and Windows COM API code, and cover the full range from single parts to assemblies. General-purpose multimodal models fall short on these tasks, while single-task fine-tuning is too narrow to support the diverse calls that upper-layer agents issue. We build **IndustryForge-27B** on top of Qwen3.5-VL-27B by curating and integrating six industrial-CAD sub-corpora totalling ~52k multimodal samples—CAD Visual QA (CAD-VQA), parametric CAD code (text2cadquery), assembly-level CAD code (text2cadquery-assembly), and three COM sub-corpora for Inventor / SolidWorks (com_2d / com_3d / com_assembly)—and training with a unified multi-task SFT recipe. Across four CAD-domain benchmarks IndustryForge-27B lifts the base model by +33.65 pp on average and *outperforms* the strong closed-source model gpt-5.4 on all four; across eleven general-capability benchmarks it retains, and slightly improves upon, the base model (+1.56 pp mean, no catastrophic forgetting). IndustryForge-27B *will* serve as the common substrate for downstream industrial-agent projects, providing a unified starting point for a full-stack industrial agent that spans from CAD design to industrial-software operation, from parts to assemblies, and from single-shot generation to closed-loop self-improvement.

 **Date:** July 31, 2026

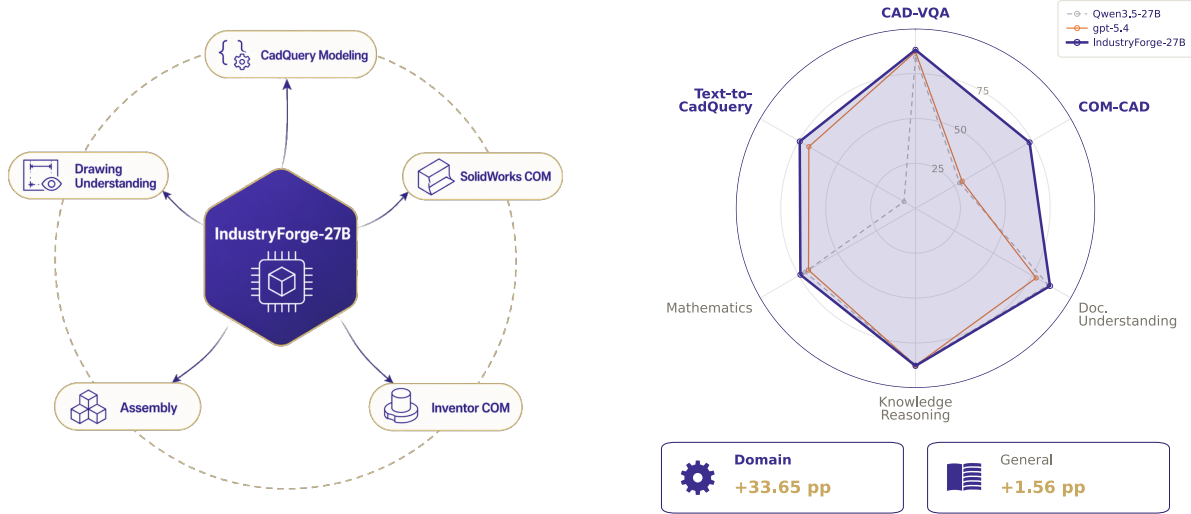


Figure 1: **IndustryForge-27B at a glance.** *Left — Capability Radiation.* Five industrial-CAD skills (Drawing Understanding, CadQuery Modeling, SolidWorks COM, Inventor COM, Assembly) radiate from the IndustryForge-27B badge and share a common “full-stack CAD loop” dashed orbit that visualises the end-to-end *design* → *modeling* → *assembly* → *software-operation* pipeline the substrate supports. *Right — Six-axis radar.* Three domain axes (upper half: CAD-VQA, Text-to-CadQuery, COM-CAD) and three aggregated general-capability axes (lower half: Doc. Understanding, Knowledge Reasoning, Mathematics) compare IndustryForge-27B (blue-purple, filled) against Qwen3.5-27B (grey, dashed) and gpt-5.4 (orange). Two floating cards report the headline: IndustryForge-27B wins 4/4 CAD-domain benchmarks against gpt-5.4 (+33.65 pp mean) while retaining general capabilities on 11 public benchmarks (+1.56 pp mean, no catastrophic forgetting).

1 Introduction

1.1 Why industrial design and manufacturing need a specialised foundation model

Computer-Aided Design (CAD) sits at the starting point of virtually every modern manufacturing pipeline: from a single screw to an entire aircraft, engineers carve parts, assemblies, and engineering drawings out of parametric modelling software one operation at a time. In the past two years, the rise of *programmable CAD* [1, 2] together with the rapid improvement of large language models on code generation has made it feasible to obtain executable CAD scripts from a natural-language description or a single rendered image [3, 4]. Nevertheless, once we push beyond “a program that runs” towards “a manufacturable, assemblable engineering artefact”, general-purpose multimodal foundation models degrade sharply along three axes:

- **“Reads the drawing, writes the wrong code”.** A general multimodal model can caption an isometric screenshot in prose that sounds plausible, yet when the same content has to be translated into CadQuery or OpenSCAD scripts, dimensions, constraints, and coordinate frames are frequently mis-assigned—the shape is qualitatively right but the numbers are entirely wrong.
- **Poor coverage of industrial APIs.** Windows-COM interfaces exposed by Inventor, SolidWorks, and AutoCAD occupy a vanishing share of public code corpora. General models have essentially never seen how to *extrude a feature via COM*, let alone how to assemble parts through `InsertMoveCopyBody2` and mate constraints.
- **Absence of the assembly scale.** Public CAD data are overwhelmingly single-part; assembly-level constraints, coaxial-alignment, and interference checking are chronically under-represented, so general-purpose models score close to zero on multi-part systems (§4.2).

None of these shortcomings can be patched by any single downstream agent alone; they must be addressed at the *foundation-model layer*. That is the starting point of this work.

1.2 Three limitations of existing routes

(1) Single-task fine-tuning. Systems in the DeepCAD / SkexGen / Text2CAD lineage [3, 5, 4] typically over-fit one narrow task—for example “text $\rightarrow$ single-part sketch sequence”—and generalise poorly across the multiple call patterns that a higher-level agent must dispatch.

(2) Prompting closed-source models directly. Using a strong closed-source model as the sole worker in a real engineering pipeline is expensive, difficult to control, and hits the coverage ceiling described above: closed-source models are dramatically weaker on Inventor / SolidWorks COM code, and near-zero on assembly-level tasks (gpt-5.4 scores 3.85% on CadQuery Assembly in §4.2).

(3) No unified foundation. Capabilities within the CAD ecosystem—from part to assembly, from CadQuery to COM—are currently scattered across specialised subsystems: IterCAD targets closed-loop generation, AssemCAD targets assemblies, ComAct targets COM operations, and SimLoop targets the CAD-CAE loop. These subsystems share no common *substrate*, forcing each team to independently solve the prerequisite question “can our foundation model read CAD drawings and write CAD code at all?”—a large and duplicated up-front cost.

1.3 Our stance: the foundation model as common substrate

IndustryForge-27B does not attempt to solve every industrial-agent problem end-to-end. Instead, we abstract four *shared skills* out of the industrial-CAD landscape and pre-install all four into a single 27B-parameter multimodal model via multi-task SFT: (i) **CAD visual understanding**, (ii) **CadQuery parametric code generation**, (iii) **COM API code generation** for Inventor / SolidWorks / AutoCAD, and (iv) **assembly-level CAD code generation**. Concretely:

IndustryForge-27B is a common substrate for industrial agents, not an end-to-end CAD generation system.

Figure 1 summarises the resulting positioning. The *left* panel radiates the four shared skills as five capability tags—Drawing Understanding, CadQuery Modeling, SolidWorks COM, Inventor COM (COM is split by target application), and Assembly—out of the IndustryForge-27B badge, all sharing a common “full-stack CAD loop” orbit that visualises the end-to-end design $\rightarrow$ modeling $\rightarrow$ assembly $\rightarrow$ software-operation pipeline the substrate supports. The *right* panel reports headline scores on a six-axis radar (three domain axes plus three general-capability aggregates) against Qwen3.5-27B and gpt-5.4: IndustryForge-27B wins on every domain axis while preserving—and slightly improving—its general capabilities.

1.4 Contributions

- **An industrial multimodal SFT corpus.** We curate and integrate four task families over six sub-corpora, $\sim 52k$ samples in total (§3.2). The three COM sub-corpora share their upstream with ComAct [6], text2cadquery shares its upstream with IterCAD [7], and text2cadquery-assembly with AssemCAD [8]; the CAD-VQA sub-corpus is independently constructed by a collaborator on this project [*TODO: source and pipeline for cad_vqa to be filled in by the collaborator*]. All code samples are filtered through sandbox execution and, where applicable, Chamfer-Distance quality control.
- **A training recipe for unbalanced multi-task SFT.** On top of Qwen3.5-VL-27B we combine LoRA, DeepSpeed ZeRO-3, sequence-parallel = 8, and padding-free packing (§3.3) with per-subset validation, producing a v4.1 configuration that trains stably on $8 \times A100$ -80 GB.
- **A systematic evaluation suite for CAD code generation.** We put together a four-benchmark domain suite (CAD VQA, CadQuery, COM CAD, CadQuery Assembly) and an eleven-benchmark general suite (aime26, arc, chartqa, gsm8k, gsm8k_v, math_vision, mmlu, mmmu, ocr_bench, science_qa, trivia_qa), and report head-to-head comparisons against Qwen3.5-27B and gpt-5.4 (§4).
- **An open industrial-agent substrate.** We release IndustryForge-27B checkpoints, which *will* serve as the shared starting point for the IterCAD, AssemCAD, ComAct, and SimLoop industrial-agent projects.

2 Related Work

2.1 Programmatic CAD

The premise of *programmatic CAD* is to describe a 3D model with executable code rather than GUI clicks. CadQuery [1], OpenSCAD [2] and Onshape FeatureScript are representative frameworks in this line: an “extrude”

button is replaced by an equivalent Python or DSL statement, so that the entire modelling process becomes version-controllable, parametric, and automatable. This paradigm shift is especially favourable for large language models—they no longer need to imitate pixel-level GUI interaction and can instead work in their strongest medium, the code space. The CadQuery portion of IndustryForge-27B rests directly on this code-driven CAD assumption.

2.2 Text- and Image-to-CAD

From early systems that emit sketch sequences (DeepCAD [3], SkexGen [5]), to more recent work whose core task is “natural language $\rightarrow$ CAD code or parameter sequence” (Text2CAD [4], CAD-LLM [9]), Text- and Image-to-CAD has been an active research direction. In the past year, as code LLMs have matured, the sub-line that generates CadQuery / OpenSCAD code directly has attracted substantial attention. On top of this, a family of “CAD agents” aimed at real engineering pipelines has emerged:

- **IterCAD** [7] lifts CAD code generation from a single-shot mapping into a “generate $\rightarrow$ verify $\rightarrow$ refine” multi-round loop, combining compiler feedback, sandbox execution, and visual critique so that a 4B model can outperform closed-source giants such as GPT-5 on error rate and AUC-TR.
- **AssemCAD** [8] targets Production-Ready Assembly Generation: it decomposes mechanical assemblies into “What to Build / How to Build / How to Verify” stages, letting the semantic reasoning of LLMs work in concert with a deterministic CAD kernel.
- **ComAct** [6] introduces the *COM-as-Action* paradigm: rather than have an agent stare at pixels and click buttons, ComAct operates SolidWorks / Inventor / AutoCAD’s internal objects through COM code, paired with the ComCADBench benchmark and the ComForge parallel-execution platform for agent training.

These works each tackle a different slice of the CAD-agent problem, but they share one prerequisite: *the underlying model must first be able to read CAD-relevant vision and to write at least one form of CAD code*. IndustryForge-27B is designed as the “qualified starting point” for exactly these downstream agents.

2.3 Industrial software: the COM / API ecosystem

Within the Windows ecosystem, the Component Object Model (COM) is a system-level object model adopted by a large swath of traditional heavy professional software—Autodesk Inventor, Dassault SolidWorks, Autodesk AutoCAD, Microsoft Office, Adobe Suite, and many more. Compared to GUI click chains or fragmented REST APIs, COM offers a much closer approximation of the software’s “native language”: an agent directly manipulates sketches, parts, and assemblies as internal objects, rather than emulating a mouse. COM corpora, however, are extremely scarce on the public internet—most publicly available tutorials cover only Office Automation, while CAD-specific COM interfaces remain effectively locked away behind closed proprietary documentation and internal scripts. Through three ComAct-aligned COM sub-corpora—`com_2d`, `com_3d`, `com_assembly`, totalling 30k samples—IndustryForge-27B substantially alleviates this data scarcity.

2.4 Multimodal foundation models

The Qwen-VL series (Qwen-VL, Qwen2-VL, Qwen3-VL, Qwen3.5-VL) [10, 11] and the InternVL series [12] are representative open-source multimodal foundation models of the past two years. Medicine, coding, and mathematics all offer mature examples of domain SFT / CPT on such foundations. IndustryForge-27B follows this “general multimodal foundation + domain SFT” recipe, extending it from the text-code domain into the compound modality of *industrial-CAD vision + code*.

Positioning of this work. IndustryForge-27B sits at the *foundation-model layer* between general multimodal foundation and industrial-agent systems—deeper than DeepCAD / Text2CAD, because it is not tied to any single downstream task, yet more specialised than closed-source generalists, because it pre-injects CAD vision, CadQuery, COM, and assembly capabilities.

3 Methodology

3.1 Overview

The construction of IndustryForge-27B decomposes into three stages: *data $\rightarrow$ SFT $\rightarrow$ downstream agents*. Figure 2 illustrates the resulting three-tier structure.

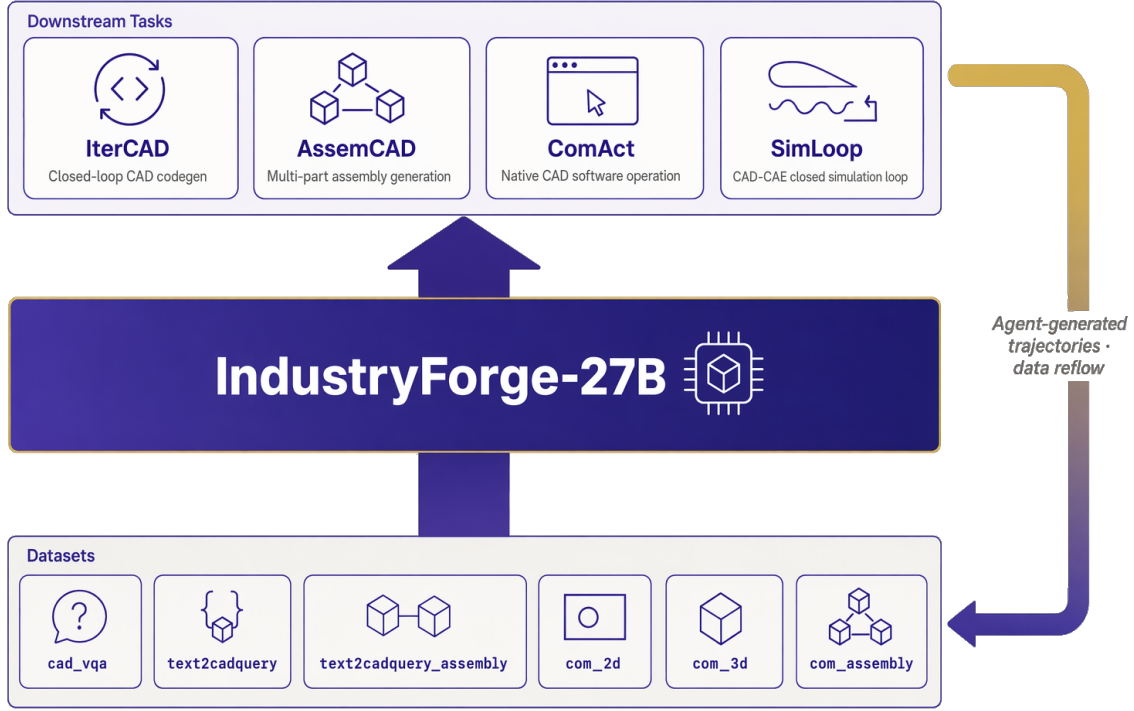


Figure 2: **Framework overview.** A vertical three-tier stack. *Bottom:* six domain sub-corpora (CAD-VQA 4k, text2cadquery 17k, text2cadquery-assembly 1k, com_2d 20k, com_3d 5k, com_assembly 5k), colour-coded by their upstream project (IterCAD / AssemCAD / ComAct / independent). *Middle:* the IndustryForge-27B foundation model, itself built on Qwen3.5-VL-27B and trained on ~ 52 k samples. *Top:* four downstream agents that will consume the substrate—IterCAD (closed-loop generation), AssemCAD (assembly generation), ComAct (COM operation), and SimLoop (CAD-CAE loop). The main flow (wide arrow) is bottom-up. A thin *return* arrow on the right marks a planned data-feedback path: high-quality traces produced by downstream agents in real tasks can be recycled into the substrate’s next training round.

3.2 Training data construction

This section is the heart of the report. We organise the discussion around four task families over six sub-corpora: CAD visual understanding, parametric CAD code generation, assembly-level CAD code generation, and industrial-API (COM) code generation.

3.2.1 CAD visual understanding: CAD-VQA-4k

Objective. Inject a “CAD-reading” prior into the foundation model. Multi-view engineering drawings, isometric 3D renders, and structured dimensional annotations occupy a negligible share of general-purpose pre-training corpora, so the model must be explicitly taught to read the geometry and drafting semantics carried by CAD-specific visual representations.

Source and construction. First, complex CAD drawings containing part drawings, assembly drawings and multi-view information are screened from raw mechanical drawings. In the question design phase, typical exam questions from university mechanical engineering courses including Mechanical Manufacturing Technology, Tolerances and Technical Measurements, and Fundamentals of Mechanical Design are systematically sorted out. Based on these materials, task categories are summarized, such as basic dimension reading, machining process planning, operating principles of mechanisms, assembly tolerance analysis, and machining error calculation. Afterwards, a multi-modal model is adopted to generate corresponding questions, standard answers and explanations. A review model is then used to verify the rationality of questions and the correctness of answers; unqualified samples are eliminated or revised. Finally, a structured CAD-VQA dataset is constructed, which includes images, questions, answers, explanations and question type labels.

3.2.2 Parametric CAD code generation: text2cadquery-17k

Objective. Teach the model to translate a natural-language description or a single-view render into executable **CadQuery** Python code that renders to the target geometry under a deterministic kernel.

Source and construction. This sub-corpus is co-sourced with the D_{S1} (drawing $\rightarrow$ code) stage of IterCAD’s expert trajectories. Raw annotations, drawn from normalised scripts in a programmatic-CAD library, pass through the following pipeline to yield 17k high-quality samples: (1) sandbox execution verifies that every script runs and renders; (2) a Chamfer-Distance threshold filters out samples that “run but drift away from the intended geometry”, suppressing annotation noise; (3) each script is automatically paired with multi-view renders and a natural-language description, forming a complete (multimodal input $\rightarrow$ code) QA pair; and (4) the whole set is deduplicated against every downstream benchmark at both UUID and structural levels (§3.2.6). A more complete write-up of the underlying data curation is available in the IterCAD paper (§ “Data Curation”).

3.2.3 Assembly-level CAD code generation: text2cadquery-assembly-1k

Objective. Push CAD code generation from “single part” to “assembly”. A real mechanical system is made of multiple parts glued together by engineering-level constraints, coaxial alignments, and interference checks. A runnable piece of CAD code is far from sufficient to guarantee an assemblable mechanical system.

Source and construction. This sub-corpus is co-sourced with the assembly data of AssemCAD. Construction follows AssemCAD’s What / How / Verify three-stage recipe: (1) a large model first parses a natural-language assembly requirement into a structured *Assembly Specification* that explicitly encodes the parts, interfaces, and assembly relations; (2) parametric part geometry and an assembly script are generated from the Specification; and (3) geometric interference checks, assembly connectivity analysis, and engineering-rule validation gate the output at the system level. Only assemblies that clear these checks enter the SFT set. Because the acceptance bar is high, the final size lands at $\sim 1k$ —small in count, but every sample carries an assembly-level geometric-consistency guarantee. The full construction and verification pipeline is described in the AssemCAD paper (Design Understanding and Assembly Construction).

3.2.4 Industrial-software API code generation (the COM family)

Objective. Equip the substrate with the ability to operate mainstream industrial CAD software (SolidWorks / Inventor / AutoCAD), and to do so *not* through GUI click chains but through COM code that directly invokes the software’s internal objects—the *COM-as-Action* paradigm proposed by ComAct.

The three sub-corpora cover:

- **com_2d-20k:** 2D sketching and drafting COM operations (sketch constraints, dimensioning, drawing templates).
- **com_3d-5k:** 3D part-modelling COM operations (sketch $\rightarrow$ extrude / revolve / sweep / loft feature construction).
- **com_assembly-5k:** assembly-level COM operations (InsertMoveCopyBody2, mate constraints, interference checks).

Source and construction. All three sub-corpora are co-sourced with data produced by ComAct, though we retain only the “code IO” portion for SFT (dropping the agent-trajectory labels used by ComAct itself). Every raw COM script has been *sandbox-executed* in the **ComForge** parallel-execution platform—each ran to completion inside a containerised Windows VM instance of SolidWorks / Inventor / AutoCAD, produced concrete CAD artefacts (.stl / .step), and only samples whose execution succeeded *and* whose artefacts passed structural checks were kept. Data curated this way is far superior to the public-internet COM scraps on both syntactic runability and engineering-semantic correctness. A more complete write-up of the COM-as-Action motivation and the ComForge platform is available in the ComAct paper.

3.2.5 Data statistics

Table 1 aggregates the six sub-corpora into four task families (CAD-VQA, CadQuery Parts, COM Parts, Assembly), matching the inner-ring taxonomy of Figure 3, and Figure 3 shows both levels of aggregation—outer ring per sub-corpus, inner ring per task family—sliced by sample count.

Table 1: **Training data statistics.** Four task families over six sub-corpora, $\sim 52k$ samples in total. Rows follow the inner-ring ordering of Fig. 3; sub-corpus-level breakdowns appear in parentheses inside the “Sub-corpora” column.

Task family	Sub-corpora	Size	Primary modality	Avg. tokens	Upstream source
CAD-VQA	cad_vqa	4 k	multi-view + text	med	Independent [TODO]
CadQuery Parts	text2cadquery	17 k	single-view + text	med-L	IterCAD
COM Parts	com_2d (20 k) + com_3d (5 k)	25 k	UI shot + text	long	ComAct
Assembly	text2cadquery-assembly (1 k) + com_assembly (5 k)	6 k	multi-view / UI + text	long	AssemCAD / ComAct
Total	—	$\sim 52k$	—	—	—

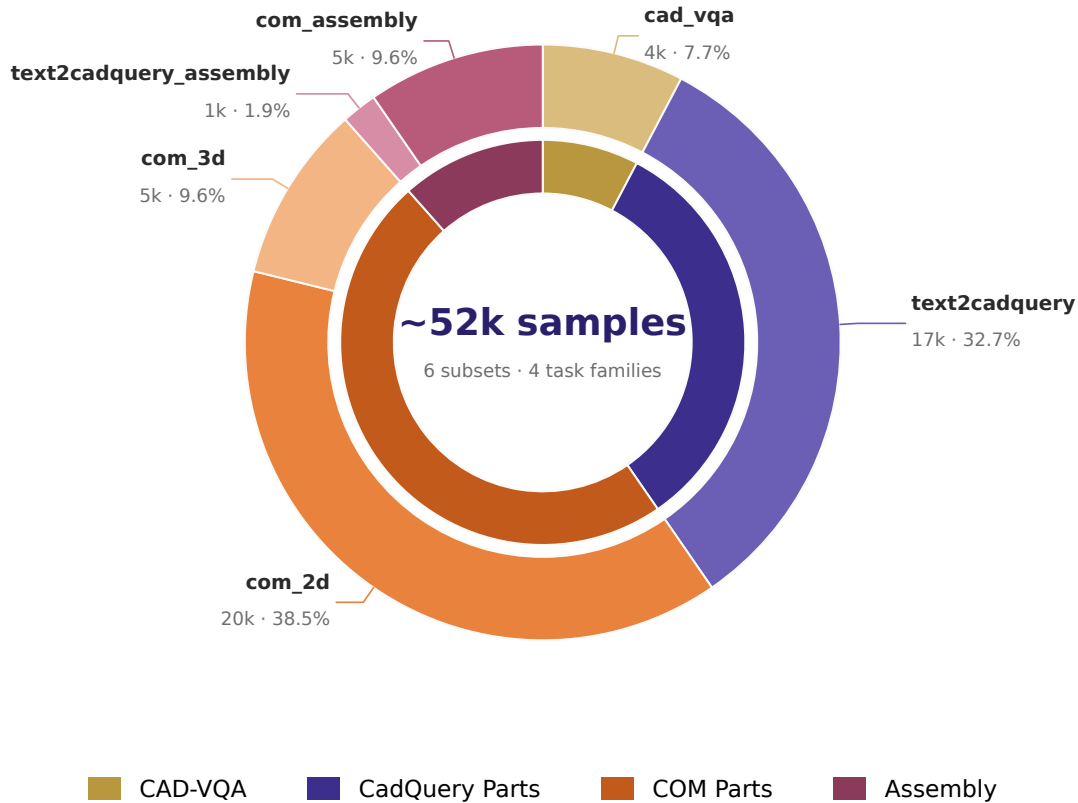


Figure 3: **Data distribution.** Outer ring: six sub-corpora sliced by sample count; consecutive slices belong to the same inner-ring family, giving visual alignment between the two levels. Inner ring: four task families (CAD-VQA, CadQuery Parts, COM Parts, Assembly). Two takeaways: (1) the COM family (com_2d + com_3d) is the largest single-part contributor by sample count; (2) the Assembly family is small in count (6 k) but each sample is long, so it contributes disproportionately to real training cost.

3.2.6 Quality control and decontamination

We apply two levels of decontamination to all six sub-corpora: **(1) UUID-level.** Every sub-corpus is precisely deduplicated against every downstream benchmark (Zero-To-CAD, com_QA_mini, the CadQuery Assembly evaluation set) by original-sample UUID—closing direct train/test leakage. **(2) Structural-level.** For CadQuery-related sub-corpora we further hash on the AST-normalised script, closing trivial-variant leakage (variable renaming, formatting differences) that would slip past UUID-level dedup. The COM sub-corpora additionally apply a Chamfer-

Distance nearest-neighbour filter on the final CAD artefacts, keeping only samples with distinct geometry to preserve coverage.

3.3 Training recipe

3.3.1 Base model choice

We adopt **Qwen3.5-VL-27B** [11] for three reasons: (1) the Qwen-VL family sits reliably in the top tier of public multimodal baselines and its visual encoder handles grey-scale engineering line-work adequately; (2) a 27B parameter budget balances capability against single-node $8 \times A100$ -80 GB deployability, which downstream agents can run uniformly; and (3) Qwen3.5-VL’s visual tokeniser gives us fine control over image-token count via `IMAGE_MAX_TOKEN_NUM`, providing stable support for multi-view inputs.

3.3.2 SFT rather than CPT

The domain data already carries explicit QA / code pairs—the target side is well-defined—so direct SFT is the more economical path. CPT is better suited to scenarios where the target-language distribution itself must migrate (e.g. bilingual CPT); this work concentrates on “English code + mixed-language descriptions $\rightarrow$ more structured CAD/COM code”, a typical *instruction-tuning-solvable* problem. From v1 onwards we therefore run pure SFT.

3.3.3 Key engineering settings

Sequence parallel (`sequence_parallel_size = 8`). Long-sequence samples are sliced across 8 GPUs. CAD and COM scripts are much longer than typical instruction samples (`com_assembly` routinely exceeds 5k tokens), and sequence parallel is the key to stable long-sample training. In early ablations, disabling it produced marked instability and memory pressure on long samples.

DeepSpeed ZeRO-3. Optimiser states, gradients, and parameters are sharded within the data-parallel group. In conjunction with sequence parallel, a 27B multimodal SFT fits stably into a single $8 \times A100$ -80 GB node.

Multi-task sampling ratio. The six sub-corpora are mixed in a *flat* proportion (no hand-tuned re-weighting). This is the simplest and most stable multi-task strategy.

Validation strategy. Each sub-corpus maintains its own validation split, and we evaluate every sub-corpus independently during training. This prevents smaller sub-corpora (especially `text2cadquery-assembly` and `CAD-VQA`) from being diluted by the larger ones, and surfaces over- or under-fitting on the small tails early.

4 Experiments

4.1 Evaluation protocol

We partition evaluation into a **domain** suite and a **general** suite, matching the multi-task structure of training.

Domain suite (4 benchmarks).

- **CAD VQA** — visual question answering over engineering drawings and 3D screenshots. Metric: QA accuracy.
- **CadQuery** — parametric CAD code generation on single-part tasks (the Zero-To-CAD series). Metric: `passed@1e-3`, i.e. the fraction of samples whose generated code executes in the sandbox *and* whose resulting geometry has a Chamfer Distance $\leq 10^{-3}$ against the ground truth.
- **COM CAD** — Inventor / SolidWorks API code generation on the `com_QA_mini_0311` evaluation set (60 samples). Metric: end-to-end pass rate of the final CAD artefact (sandbox execution + structural geometry match).
- **CadQuery Assembly** — assembly-level CAD code generation, sharing the evaluation set with `AssemCAD`. Same metric as `CadQuery` but with a higher bar: all parts of the assembly must match.

General suite (11 benchmarks). `aime26`, `arc`, `chartqa`, `gsm8k`, `gsm8k_v`, `math_vision`, `mmlu`, `mmmu`, `ocr_bench`, `science_qa`, `trivia_qa`. Coverage spans mathematics, multimodal reasoning, charts, OCR, science QA, and commonsense. Each benchmark’s primary metric follows its published protocol (accuracy or pass rate).

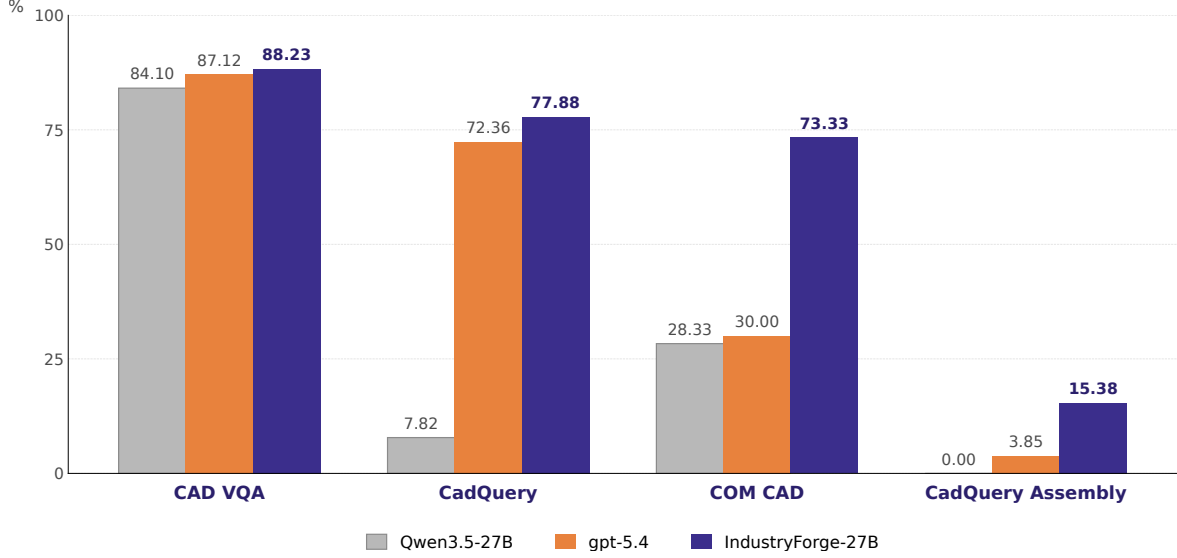


Figure 4: **Domain benchmark results.** Grouped bars per benchmark: Qwen3.5-27B (grey), gpt-5.4 (orange), IndustryForge-27B (blue-purple). Value labels above each bar. Mean = 63.71%, +33.65 pp over the base model, and IndustryForge-27B wins on 4/4 benchmarks against gpt-5.4.

Comparison models. Qwen3.5-27B (the untouched base model, serving as the “no-change baseline”); gpt-5.4 (a strong closed-source frontier model representing “the strongest publicly consumable general model at the time of writing”); and IndustryForge-27B (this work). gpt-5.4 was not evaluated on the general suite, so those comparisons are two-way.

Evaluation infrastructure. All four domain benchmarks share the same `swift_eval` sandbox-execution and Chamfer-Distance pipeline; predictions, reviews, and aggregated reports live under `swift_eval/results/{predictions,reviews,reports}/{model}/{bench}...`. Aggregation is driven by `stat_cd.py` and result management by `btool.py`, which together keep results consistent across renames and re-runs.

4.2 Main results: domain benchmarks

Figure 4 summarize the domain results.

Key observations.

- **An order-of-magnitude jump on CadQuery.** 7.82% $\rightarrow$ 77.88% (+70.06 pp), *overtaking* gpt-5.4 (72.36%)—strong evidence that domain SFT has enormous leverage on parametric CAD code generation.
- **Crushing the closed-source model on COM CAD.** 28.33% $\rightarrow$ 73.33% (+45.00 pp), and +43.33 pp ahead of gpt-5.4. COM code is exactly where public data are scarcest, and it is also where the closed-source model shows the biggest gap—making this the largest source of our differentiation.
- **First usable non-zero assembly-level pass rate.** Both the base model and gpt-5.4 sit in the near-total-failure regime (0.00% / 3.85%). IndustryForge-27B reaches 15.38%—to our knowledge, the first releasable industrial-CAD foundation with a non-zero usable pass rate on assembly-level generation.

4.3 Main results: general benchmarks

Table 2 presents the 11-benchmark general suite, grouped by capability area (Mathematics / Knowledge Reasoning / Document Understanding) and, within each group, alphabetically. Δ against the base model is discussed in the prose analysis rather than tabulated.

Across the 11 general benchmarks, IndustryForge-27B improves the base model on 7 with a mean +1.56 pp—no catastrophic forgetting. Notably, the capabilities most correlated with what our CAD/COM corpus emphasises—*multimodal geometric reasoning*, *structured document understanding*, and *long-context / multi-step consistency*—all see positive spillover:

Table 2: **Main results on general benchmarks.** Best cells in bold. Groups: Mathematics / Knowledge Reasoning / Document Understanding.

Benchmark	Qwen3.5-27B	gpt-5.4	IndustryForge-27B
<i>Mathematics</i>			
aime26	83.33%	66.67%	83.33%
gsm8k	96.29%	97.60%	97.35%
gsm8k_v	57.24%	44.60%	52.99%
math_vision	47.33%	66.20%	62.30%
<i>Knowledge Reasoning</i>			
arc	95.15%	95.20%	95.30%
mmlu	90.14%	92.77%	90.95%
mmmu	78.33%	77.00%	78.56%
science_qa	95.99%	94.80%	94.84%
trivia_qa	75.88%	78.00%	79.12%
<i>Document Understanding</i>			
chartqa	80.56%	76.20%	84.40%
ocr_bench	90.30%	78.90%	88.60%
Mean (11)	81.86%	78.90%	83.42%

- **math_vision +14.97 pp**, the single largest gain. **math_vision** draws heavily on “read three views / spatial diagrams → reason about length / area / angle / volume”—a mental process highly homologous to CAD-VQA’s “three views → structured dimension QA” and to text2cadquery’s “single- or multi-view → parametric geometry script”. The internal “multi-view → geometry” representation acquired during SFT transfers directly to **math_vision**-style geometric-visual reasoning.
- **chartqa +3.84 pp**. CAD engineering drawings embed dense annotations and tabular dimension lists whose visual structure is homologous to chart QA. The model’s SFT-acquired ability to “read numbers from compact figure/table structure” spills over onto chart-QA.
- **trivia_qa +3.24 pp and gsm8k +1.06 pp**. CAD and COM code corpora are archetypal *long-context, multiple variable bindings, structured output* samples; they strengthen the model’s grip on long-context consistency and multi-step symbolic stability, which shows up as light but consistent gains on tasks that demand cross-sentence factual and numerical consistency.
- **mmlu +0.81 pp, mmu +0.23 pp, arc +0.15 pp**. Three knowledge-reasoning benchmarks all register small positive gains. The absolute magnitudes are modest but they demonstrate that injecting CAD / COM data does *not* squeeze out the model’s existing academic and commonsense knowledge—a direct dividend of a conservative multi-task-SFT recipe (LoRA, frozen visual tower).

Taken together, the seven positive shifts cluster along three axes—*geometric visual reasoning*, *structured figure-text understanding*, and *long-context / multi-step reasoning*. They are not a specialisation-for-generalisation trade-off but a secondary spillover naturally carried by the design of a CAD / COM SFT corpus, and lend side-evidence to our decision to pre-install “vision + structured code” capabilities into the foundation-model layer.

4.4 Case studies

We close the experiments with five qualitative case studies that make the domain / general story concrete (Figure 5-Figure 9). Each case contrasts Qwen3.5-27B, gpt-5.4, and IndustryForge-27B on the same input.

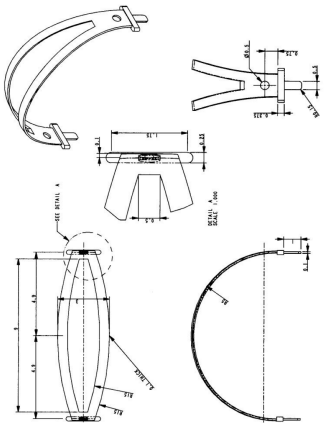
5 Conclusion

Starting from Qwen3.5-VL-27B, this work performs multi-task SFT on six industrial-domain multimodal sub-corpora (~52k samples in total) to produce IndustryForge-27B—a foundation model for industrial agents that covers four core skills: CAD visual understanding, parametric CAD code generation, assembly-level CAD code generation, and COM industrial-software code generation. On the four CAD-domain benchmarks the model averages +33.65 pp over its base and wins 4/4 against gpt-5.4; on 11 general-capability benchmarks it averages +1.56 pp with no catastrophic forgetting.

IndustryForge-27B is *not* an end-to-end CAD generation system, but the *common substrate* for industrial-agent projects. It settles the prerequisite question “can our foundation model read CAD drawings and write CAD code at

Case 1 - CAD-VQA — dimension reading on a multi-view drawing

Input drawing + question



What is the total length of the part
(the distance between the two end pieces)?

Target: 9.8

Qwen3.5-27B FAIL

"The vertical dimension line on the left has an overall label of 9, split into two 4.9 segments. But $4.9 + 4.9 = 9.8$, which contradicts the 9 label. Let me re-read the digits... the total label is 9."

Final answer: 9

gpt-5.4 FAIL

"From the front view, the total length is labeled as 9 (the vertical dimension on the left)."

Final answer: 9

IndustryForge-27B PASS

1. In the front view, a vertical dimension line marks the total length.
2. The dimension line is split into two segments, each labeled 4.9.
3. Total = $4.9 + 4.9 = 9.8$.

Final answer: 9.8

Figure 5: **Case 1 — CAD VQA (part identification / dimension reading)**. Prose-style QA layout: an input image and question on the left; three stacked answer rows on the right showing each model’s chain-of-thought excerpt and final answer with PASS / FAIL badges. IndustryForge-27B correctly reads the multi-view geometry and matches the target answer; the base model and gpt-5.4 mis-count features.

all?” once at the foundation-model layer, so that IterCAD, AssemCAD, ComAct, and SimLoop all *will* start from the same launchpad. Because none of these downstream projects has yet formally integrated the substrate, we use the future tense throughout this report when describing that role.

Limitations.

- Single-part samples still dominate the training distribution; assembly-level pass rate, though lifted from 0 to a usable non-zero (15.38%), has substantial room to improve.
- The current scope only covers CAD code generation and visual understanding; simulation and CAE are not modelled at the foundation layer, and depend on SimLoop’s CAD-CAE loop to close that gap.
- This report covers only SFT—closed-loop RL is deferred to the IterCAD line of work on top of the substrate.
- The COM sub-corpora focus on the “Windows CAD trio” (SolidWorks, Inventor, AutoCAD). Other COM domains (Office, Adobe) are not yet included.

Case 2 · Text-to-CadQuery — parametric CAD code from a natural-language description

Task description + ground truth

Create a coordinate system with Euler angles (0, 0, -90)° and translation (0, 0.3, 0). Draw a closed loop through: (0, 0) → (0.75, 0) → (0.75, 0.15) → (0.3, 0.15) → (0.3, 0.45) → (0, 0). Apply the rotation + translation, then extrude 0.3 along the normal.
Final size: 0.75 × 0.45 × 0.3



Ground truth STL
Target: $CD \leq 1 \times 10^{-3}$

Qwen3.5-27B CadQuery execution failed FAIL

```
import cadquery as cq
wp = (cq.Workplane()
      .rotate(0, 90, 0)
      .move(0, 0.3, 0)) # ← arity error
sketch = (wp.moveTo(0, 0)
          .line(0.75, 0).line(0, 0.15)
          .line(-0.45, 0).line(0, 0.3)
          .close())
r = sketch.extrude(0.3)
```

RuntimeError: Workplane.move() takes 1-3 args, got 4

gpt-5.4 $CD = 7.42 \times 10^{-3}$ ($> 1 \times 10^{-3}$) FAIL

```
import cadquery as cq
plane = cq.Plane(origine=(0, 0.3, 0),
                 xDir=(0, -1, 0),
                 normal=(0, 0, 1))
r = (cq.Workplane(plane)
     .moveTo(0, 0).lineTo(0.75, 0)
     .lineTo(0.75, 0.15).lineTo(0.3, 0.15)
     .lineTo(0.3, 0.45)
     .lineTo(0, 0) # ← skips (0, 0.45)
     .close()).extrude(0.3)
```



IndustryForge-27B $CD \approx 0$ (passed@1e-3) PASS

```
import cadquery as cq
cs = cq.Workplane("front").transformed(
    rotate=(0, 0, -90),
    offset=(0, 0.3, 0))
sketch = (cs.moveTo(0, 0)
          .lineTo(0.75, 0).lineTo(0.75, 0.15)
          .lineTo(0.3, 0.15).lineTo(0.3, 0.45)
          .lineTo(0, 0.45) # correct L-shape
          .close())
r = sketch.extrude(0.3)
```

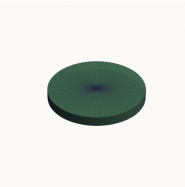


Figure 6: **Case 2 — Text-to-CadQuery (single part)**. Left card: task description, ground-truth STL preview, and $CD \leq 10^{-3}$ target. Right: three stacked rows with per-model code excerpts (the discriminating lines highlighted) and the corresponding STL renders. IndustryForge-27B produces a CadQuery script that passes at 10^{-3} ; the base model errors out on the sketch API and gpt-5.4 mis-dimensions the target feature.

Case 3 · COM SolidWorks — parametric API code targeting the SolidWorks COM interoper

Task description + ground truth

Create a coordinate system at the origin with zero Euler angles. Draw a circle on the front plane: center (0.375, 0.375), radius 0.375. Extrude along the normal by 0.075. Export the resulting solid as STL to C:/Users/Docker/Downloads/00119372.stl
Final size: 0.75 × 0.75 × 0.075



Ground truth STL
Target: $CD \leq 1 \times 10^{-3}$

Qwen3.5-27B COM Type mismatch (SaveAs3 arity) FAIL

```
...
def super_cast(obj, iface): ...
ext = super_cast(model.Extension,
                  'IModelDocExtension')
sketch_mgr.CreateCircleByRadius(
    0.375, 0.375, 0, 0.375)
feat_mgr.FeatureExtrusion3(True, ..., 0.075)
ext.SaveAs3(stl, 0, 1, None, None, 0)
# ← missing Warnings BYREF arg (6/7)
...
```

pywintypes.com_error: SaveAs3 Type mismatch (6 args, 7)

gpt-5.4 COM Type mismatch (raw COMObject) FAIL

```
...
sw = Dispatch('SldWorks.Application')
model = sw.NewDocument(tpl, 0, 0, 0)
model = win32com.client.Dispatch(model)
ext = model.Extension
# ← raw COMObject, no super_cast
ok = ext.SelectByID2('Front Plane', 'PLANE',
                    0, 0, 0, False, 0,
                    None, 0) # ← Type mismatch
...
```

pywintypes.com_error: SelectByID2 Type mismatch on None arg

IndustryForge-27B $CD = 5.82 \times 10^{-5}$ (passed@1e-3) PASS

```
...
def super_cast(obj, iface):
    q = obj._oleobj_.QueryInterface(
        getattr(swMod, iface).CLSID,
        pythoncom.IID_IDispatch)
    return getattr(swMod, iface)(q)
ext = super_cast(model.Extension,
                  'IModelDocExtension')
ext.SelectByID2('Front Plane', ...)
ext.SaveAs3(stl, 0, 1, None, 0, 0) # ← 7 args
...
```

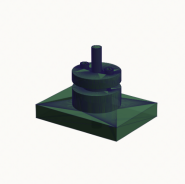


Figure 7: **Case 3 — COM (SolidWorks)**. A single-cylinder task executed through the SolidWorks COM API. Each row shows the model’s key COM calls (sketch → extrude → move body) and the resulting STL. IndustryForge-27B uses the correct helper structure and passes; gpt-5.4 mis-selects the extrude direction; the base model fails at COM plane construction.

Case 4 · CadQuery Assembly — multi-part assembly with cq_warehouse standard parts

Task description + ground truth

Foot-mounted vertical bearing housing:
 Base 80×60×12 mm, boss ø40 × 15
 Stepped bore ø22 depth 10 in boss
 (7 mm seat + 3 mm shoulder)
 Bearing M8-22-7 seated in the bore
 Clamp cap ø40 × 8, central ø10 bore
 2× M4-0.7 SHCS at X = ±12, len 25
 Shaft ø8 × 70, concentric w/ bearing

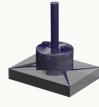


Ground truth STL
 Target: $CD \leq 1 \times 10^{-3}$

Qwen3.5-27B CD = 7.36×10^{-3} (shaft misplaced) FAIL

```
...
bearing_z = cap_top_z - 3 - 7/2 # 16.5
bearing_loc = Location(
    cq.Vector(0, 0, bearing_z))

shaft = (cq.Workplane("XY")
    .circle(4).extrude(70))
# - no centering offset
shaft_loc = Location(cq.Vector(
    0, 0, bearing_z)) # asymmetric
...
```



gpt-5.4 CD = 6.76×10^{-3} (shaft misplaced) FAIL

```
...
bearing_loc = cq.Location(
    cq.Vector(0, 0, 20))
# bearing spans Z = 20 .. 27

shaft = (cq.Workplane("XY")
    .workplane(offset=-shaft_len/2)
    .circle(4).extrude(70))
# - centered on Z = 0,
# NOT the bearing at Z = 23
...
```



IndustryForge-27B CD = 6.95×10^{-4} (passed@1e-3) PASS

```
...
bearing_center_z = (
    boss_top_z - bearing_width/2)

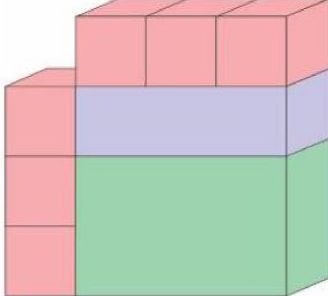
shaft = (cq.Workplane("XY")
    .workplane(offset=
        bearing_center_z
        - shaft_length/2)
    .circle(4).extrude(70))
# - symmetric about bearing axis
...
```



Figure 8: **Case 4 — CadQuery Assembly (multi-part with cq_warehouse)**. A foot-mounted vertical bearing housing with clamp cap and shaft. The discriminating detail is a single line of coordinate reasoning that centres the shaft on the bearing axis rather than on the global origin or on the bearing’s local upward extrude. Only IndustryForge-27B’s placement (`offset = bearing_center_z - shaft_length/2`) passes at 10^{-3} .

Case 5 · MATH-Vision L3 — surface-area reasoning on a glued-block solid

Input image + question



Zilda paints six cubes and two blocks separately, using 18 L in total. How much paint if she paints the structure only after gluing the parts?

Target: 11.5

Qwen3.5-27B FAIL

...Top row: 3 pink cubes. Middle: purple block. Bottom: green block...
 ...Left column: 3 cubes? 4 cubes? Total must be 6, so left column is 3...
 ...Wait, let me re-examine the structure... <repeat 23 times>
 ... * Method 2: <truncated>

Final answer: N/A

gpt-5.4 FAIL

Let cube edge = 1. Total area before gluing = $6 \cdot 6 + 22 + 14 = 72$ face-units...
 ...72 units $\Rightarrow$ 18 L, so 1 L paints 4 units...
 ...Contacts: green $\leftrightarrow$ purple (3) + purple $\leftrightarrow$ top-row (3) + left-col internal (1+1)
 + top-cube $\leftrightarrow$ purple (1) = 9 $\rightarrow$ hidden = $2 \cdot 9 = 18$...
 Exposed = $72 - 18 = 54$, paint = $18 \cdot (54/72) = 13.5$." $\Rightarrow$ \boxed{13.5}

Final answer: 13.5

IndustryForge-27B PASS

...Total area before gluing = $36 + 22 + 14 = 72$ face-units $\rightarrow$ 1 unit = 0.25 L...
 ...Contacts: left-col internal (2) + top-row internal (2) + left cubes $\leftrightarrow$ green (2)
 + top cube $\leftrightarrow$ purple (1) + green $\leftrightarrow$ purple (3) + purple $\leftrightarrow$ top-row (3) = 13...
 ...Hidden = $2 \cdot 13 = 26$, exposed = $72 - 26 = 46$...
 Paint = $46 \times 0.25 = 11.5$ liters. $\Rightarrow$ \boxed{11.5}

Final answer: 11.5

Figure 9: **Case 5 — Cross-benefit on math_vision L3**. A painted-solid surface-area puzzle. The base model runs into a “rathole” (23 rounds of “Wait, let me re-examine...” over 40k characters, the correct value 11.5 appearing at 92% of the response but never committed to \boxed{ }); gpt-5.4 under-counts contact interfaces and lands on 13.5; IndustryForge-27B enumerates all six contact groups, computes hidden = 26, exposed = 46, and commits to 11.5. This is direct evidence that CAD SFT lifts “multi-view $\rightarrow$ spatial geometry” reasoning on general benchmarks.

References

- [1] CadQuery Team. CadQuery: a python parametric CAD scripting framework based on OpenCascade, 2024. <https://github.com/CadQuery/cadquery>.
- [2] OpenSCAD Team. OpenSCAD: The programmers solid 3d cad modeller, 2024. <https://openscad.org>.
- [3] Rundi Wu, Chang Xiao, and Changxi Zheng. DeepCAD: A deep generative network for computer-aided design models. *ICCV*, 2021.
- [4] Mohammad Sadil Khan and Others. Text2CAD: Generating sequential CAD designs from beginner-to-expert level text prompts. *NeurIPS*, 2024.
- [5] Xiang Xu, Karl D. D. Peng, Karl D. D. Willis, and Others. SkexGen: Autoregressive generation of CAD construction sequences with disentangled codebooks. *ICML*, 2022.
- [6] ComAct Team. ComAct: COM-as-Action for industrial CAD software agents. *arXiv preprint*, 2026. [TODO: replace with final citation].
- [7] IterCAD Team. IterCAD: Closed-loop CAD code generation with compiler and visual feedback. *arXiv preprint*, 2026. [TODO: replace with final citation].
- [8] AssemCAD Team. AssemCAD: Production-ready assembly generation via what-how-verify decomposition. *arXiv preprint*, 2026. [TODO: replace with final citation].
- [9] Anonymous. CAD-LLM: Leveraging large language models for parametric CAD generation. *arXiv preprint*, 2024. [TODO: replace with final citation].
- [10] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-VL: A versatile vision-language model for understanding, localization, text reading, and beyond. *arXiv preprint arXiv:2308.12966*, 2023.
- [11] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibong Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, and Others. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025.
- [12] Zhe Chen, Jiannan Wu, Wenhai Wang, Weijie Su, Guo Chen, Sen Xing, Muyan Zhong, Qinglong Zhang, Xizhou Zhu, Lewei Lu, and Others. InternVL: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. *CVPR*, 2024.